

Ejemplo de uso: jurisprudencia como fuente de contexto para un sistema de consulta con IA

Este documento describe un prototipo de integración entre:

1. un sistema de consulta de documentos con IA;
2. el MCP de jurisprudencia de Corrientes; y
3. el portal público del Poder Judicial de Corrientes.

El MCP no utiliza un agente de IA para navegar el sitio. La navegación, la ejecución del desafío de Cloudflare, la captura del HTML/PDF y la extracción de texto son deterministas. La IA se utiliza únicamente en el sistema externo para interpretar la consulta del usuario, decidir qué resultados necesita y responder sobre el contexto recuperado.

Caso de uso

Los valores de resultado de este documento son ficticios y sirven únicamente para ilustrar el contrato entre la aplicación, el MCP y el modelo. No representan un fallo real ni deben usarse como fuente jurídica.

Un abogado consulta en el sistema interno:

“Buscá jurisprudencia del Superior Tribunal de Justicia de Corrientes sobre amparo contra el Organismo Previsional Demo, priorizando fallos recientes y explicá qué antecedentes podrían ser relevantes para este caso.”

El sistema no debería pedirle al modelo que navegue directamente el sitio. El flujo prototipo es:

Pregunta del usuario



Planificador de recuperación del sistema de IA

| genera filtros estructurados



MCP jurisprudencia-corrientes

| Chromium gráfico + Cloudflare + portal

|— search_jurisprudencia

|— get_jurisprudencia_detail

|— get_jurisprudencia_pdf_text



Normalizador / extractor de documentos

| metadata + texto + URL de fuente



Chunking, índice documental y/o embeddings



Contexto acotado para el modelo



Respuesta con referencias verificables

1. Configurar el MCP

El cliente MCP debe ejecutar el servidor por node directamente para no mezclar la salida de npm con los mensajes JSON-RPC. En un entorno sin display, se puede usar xvfb-run:

```
{
  "mcpServers": {
    "jurisprudencia-corrientes": {
      "command": "xvfb-run",
      "args": [
        "-a",
        "--server-args=-screen 0 1280x900x24 -nolisten tcp",
        "node",
        "/ruta/absoluta/api-remota/src/server.mjs"
      ]
    }
  }
}
```

Si el sistema que integra el MCP ya tiene un DISPLAY gráfico disponible, se puede ejecutar directamente:

```
{
  "mcpServers": {
    "jurisprudencia-corrientes": {
      "command": "node",
      "args": ["/ruta/absoluta/api-remota/src/server.mjs"]
    }
  }
}
```

Requisitos del entorno:

- Node.js 22 o superior.
- Chromium o Chrome.
- pdftotext para la extracción del texto PDF.
- xvfb-run y Xvfb si no existe un display gráfico.

2. Convertir la pregunta en filtros

El planificador del sistema de consulta puede transformar la pregunta en una llamada estructurada. La transformación puede realizarla un modelo, pero el modelo no accede al sitio ni construye HTML: solamente produce filtros para la herramienta.

Ejemplo de intención interna:

```
{
```

```
"text": "Organismo Previsional Demo",
"materias": ["Amparo"],
"anio": 2026,
"page": 1,
"perPage": 10
}
```

La aplicación llama al MCP:

```
{
  "jsonrpc": "2.0",
  "id": 10,
  "method": "tools/call",
  "params": {
    "name": "search_jurisprudencia",
    "arguments": {
      "text": "Organismo Previsional Demo",
      "materias": ["Amparo"],
      "anio": 2026,
      "page": 1,
      "perPage": 10
    }
  }
}
```

La búsqueda utiliza los filtros nativos del portal. En términos equivalentes, el MCP genera una URL de este tipo:

<https://example.invalid/jurisprudencia/fallos/fallosstj?filtros=1&texto=Organismo+Previsional+Demo&ma>

El MCP no recorre automáticamente todas las páginas. El sistema de consulta debe pedir una página concreta y aplicar su propia política de límites.

3. Resultado de búsqueda que recibe la aplicación

La respuesta contiene metadatos suficientes para seleccionar documentos y continuar la captura:

```
{
  "source": "https://example.invalid/jurisprudencia/fallos/fallosstj?filtros=1&texto=Organismo+Previsional-
  "total": 42,
  "page": 1,
  "totalPages": 5,
  "results": [
    {
      "fallo": "Sentencia 000/2099",
      "expediente": "EXP DEMO-0001/2099",
      "caratula": "PERSONA FICTICIA ALFA C/ ORGANISMO PREVISIONAL DEMO S/ AMPARO (EJEMPLO)",

```

```

    "materia": "Amparo",
    "fecha": "01-01-2099",
    "id": "99999",
    "pdfUrl": "https://example.invalid/ver-pdf/00000000-0000-4000-8000-000000000001"
  }
]
}

```

La aplicación debería conservar la respuesta completa, pero evitar cargar todos los resultados al contexto del modelo. Un criterio prototipo puede ser:

1. conservar hasta 10 resultados de la primera página;
2. priorizar coincidencia de carátula, materia y fecha;
3. recuperar el detalle y el PDF de los 3 a 5 resultados más relevantes;
4. cargar al índice documental solamente los documentos nuevos o modificados.

4. Capturar el detalle HTML

Para cada resultado seleccionado, la aplicación utiliza el id numérico devuelto por la búsqueda:

```

{
  "jsonrpc": "2.0",
  "id": 11,
  "method": "tools/call",
  "params": {
    "name": "get_jurisprudencia_detail",
    "arguments": {
      "id": 99999
    }
  }
}

```

Respuesta prototipo:

```

{
  "id": 99999,
  "source": "https://example.invalid/jurisprudencia/fallos/detalle_ajax/99999",
  "heading": "Sentencia Amparo 000/2099",
  "title": "PERSONA FICTICIA ALFA C/ORGANISMO PREVISIONAL DEMO S/AMPARO (EJEMPLO)",
  "metadata": "Expte/Legajo: EXP DEMO-0001/2099 Fecha: 01/01/2099",
  "sections": [],
  "pdfUrl": "https://example.invalid/jurisprudencia/fallos/ver_pdf_proxy/00000000-0000-4000-8000-000000000001",
  "text": "Sentencia Amparo 000/2099 ..."
}

```

El detalle puede tener secciones adicionales —sumarios, categorías, voces, normativas, votación o fallos

relacionados— según el fallo publicado. El sistema debe guardar el campo text y las secciones sin asumir que todos los documentos tienen la misma estructura.

5. Capturar el texto del PDF

Cuando el PDF sea necesario para responder, la aplicación utiliza la URL devuelta por el resultado o el detalle:

```
{
  "jsonrpc": "2.0",
  "id": 12,
  "method": "tools/call",
  "params": {
    "name": "get_jurisprudencia_pdf_text",
    "arguments": {
      "pdfUrl": "https://example.invalid/jurisprudencia/fallos/ver_pdf_proxy/00000000-0000-4000-8000-0000",
      "maxChars": 100000
    }
  }
}
```

La herramienta captura el PDF desde Chromium y ejecuta pdftotext localmente. No envía el documento a un modelo durante la captura.

Respuesta resumida:

```
{
  "source": "https://example.invalid/jurisprudencia/fallos/ver_pdf_proxy/00000000-0000-4000-8000-0000",
  "contentType": "application/pdf",
  "bytes": 746789,
  "chars": 70547,

  "text": "Dra. PERSONA FICTICIA BETA ..."
```

Si truncated es true, el prototipo no debe presentar el texto como documento completo. Puede usarlo para una primera recuperación o dejar el documento pendiente de una estrategia de paginación/extensión del MCP.

6. Normalizar y cargar el contexto documental

El sistema externo puede convertir cada resultado en un documento canónico independiente del MCP:

```
{
  "document_id": "jurisprudencia-demo:99999",
  "source": "https://example.invalid/jurisprudencia/fallos/ver_pdf_proxy/00000000-0000-4000-8000-0000",
  "source_type": "jurisprudencia_oficial",
  "court": "Superior Tribunal de Justicia de Corrientes",
```

```
"decision_type": "Sentencia",
"decision_number": "000/2099",
"date": "2099-01-01",
"case_number": "EXP DEMO-0001/2099",
"title": "PERSONA FICTICIA ALFA C/ ORGANISMO PREVISIONAL DEMO S/ AMPARO (EJEMPLO)",
"subject": "Amparo",
"retrieved_by": "mcp-jurisprudencia-corrientes",
"retrieved_at": "2026-08-29T00:00:00Z",
"content": "Texto extraído del PDF..."
}
```

Antes de generar embeddings o chunks, conviene:

- conservar la URL oficial como referencia primaria;
- separar metadata de contenido jurídico;
- normalizar fechas sin perder la fecha original mostrada por el sitio;
- evitar eliminar encabezados, votos o citas normativas;
- calcular un hash del texto para detectar cambios;
- marcar si el texto fue truncado;
- registrar la consulta que produjo la captura.

Una estrategia sencilla de chunking para el prototipo es usar bloques de 4.000 a 8.000 caracteres con solapamiento de 400 a 800 caracteres, manteniendo en cada chunk:

```
{
  "document_id": "jurisprudencia-demo:99999",
  "chunk_id": "jurisprudencia-demo:99999:0003",
  "text": "...",
  "metadata": {
    "court": "Superior Tribunal de Justicia de Corrientes",
    "decision_number": "000/2099",
    "date": "2099-01-01",
    "case_number": "EXP DEMO-0001/2099",
    "source": "https://example.invalid/jurisprudencia/fallos/ver_pdf_proxy/00000000-0000-4000-8000-0000"
  }
}
```

7. Construir el contexto para la IA

El contexto final no debería ser solamente texto sin procedencia. Una plantilla mínima puede ser:

FUENTE OFICIAL: {source}
TRIBUNAL: {court}
TIPO Y NÚMERO: {decision_type} {decision_number}
FECHA: {date}
EXPEDIENTE: {case_number}
CARÁTULA: {title}

MATERIA: {subject}

CONTENIDO CAPTURADO:

{chunk_text}

El prompt del sistema de consulta puede exigir que la respuesta:

1. distinga hechos del caso, fundamentos y decisión;
2. no invente información ausente;
3. indique cuando el documento fue truncado;
4. cite el fallo mediante número, expediente y URL oficial;
5. presente la respuesta como apoyo documental y no como asesoramiento jurídico definitivo.

Ejemplo de contexto enviado al modelo:

Se encontraron antecedentes oficiales del Superior Tribunal de Justicia de Corrientes.
Usá únicamente las fuentes incluidas abajo. Si la evidencia no alcanza, indicálo.

[Documento 1]

Sentencia 000/2099 — EXP DEMO-0001/2099

PERSONA FICTICIA ALFA C/ ORGANISMO PREVISIONAL DEMO...

Fuente: https://example.invalid/jurisprudencia/fallos/ver_pdf_proxy/00000000-0000-4000-8000-00000000

{chunks recuperados del documento}

8. Política de fallos y Cloudflare

El sistema debe diferenciar estos estados:

Éxito

→ guardar resultados y fuentes.

Sin resultados

→ responder que el portal no devolvió coincidencias para esos filtros.

CHALLENGE_REQUIRED

→ no reintentar en bucle; informar que Cloudflare requiere una sesión gráfica/intervención.

Error de extracción PDF

→ conservar metadata, detalle y URL; marcar el contenido como no disponible.

Timeout

→ reintentar como máximo una vez, con espera controlada.

El MCP actual usa una sola sesión Chromium reutilizable, limita el tamaño de página y no recorre páginas automáticamente. El sistema de consulta debe aplicar límites adicionales por usuario, por consulta y por

cantidad de documentos incorporados al contexto.

9. Pseudocódigo de integración

El siguiente pseudocódigo representa la lógica de la aplicación que consume el MCP. `mcp.call_tool` es una abstracción del cliente MCP utilizado por el sistema.

```
async function recuperarJurisprudencia(pregunta) {
  // Esta etapa puede usar IA para convertir lenguaje natural en filtros.
  const filtros = await planificarFiltros(pregunta);

  const busqueda = await mcp.call_tool("search_jurisprudencia", {
    ...filtros,
    page: 1,
    perPage: 10
  });

  const candidatos = seleccionarCandidatos(busqueda.results, {
    maxDocuments: 5,
    preferRecent: true
  });

  const documentos = [];
  for (const candidato of candidatos) {
    const detalle = await mcp.call_tool("get_jurisprudencia_detail", {
      id: candidato.id
    });

    const pdf = await mcp.call_tool("get_jurisprudencia_pdf_text", {
      pdfUrl: detalle.pdfUrl || candidato.pdfUrl,
      maxChars: 100000
    });

    documentos.push(normalizar({ candidato, detalle, pdf }));
  }

  await upsertIndiceDocumental(documentos);
  return recuperarChunksRelevantes(pregunta, documentos);
}

async function responder(pregunta) {
  const contexto = await recuperarJurisprudencia(pregunta);
  return modelo.generarRespuesta({
    pregunta,
    contexto,
    instrucciones: "Citar cada afirmación con el fallo y su URL oficial. No inventar."
  });
}
```

}

En una implementación productiva, la carga al índice debería ejecutarse como proceso separado de la respuesta interactiva: primero se recuperan y persisten los documentos, luego el buscador semántico reutiliza esos documentos sin consultar el portal para cada pregunta.

10. Resultado esperado para el usuario

La respuesta del sistema de consulta podría tener esta forma:

Se identificaron antecedentes del STJ de Corrientes sobre amparos vinculados con el Organismo Previsional Demo. El resultado más reciente recuperado es la Sentencia 000/2099, expediente EXP DEMO-0001/2099, de fecha 01/01/2099. La fuente y el texto completo capturado deben revisarse antes de extraer una conclusión aplicable al caso concreto.

Fuente ficticia: [Sentencia 000/2099](#)

La IA formula la respuesta, pero la evidencia proviene del contenido capturado por el MCP y conserva la referencia oficial para auditoría humana.

Limitaciones del prototipo

- El portal puede modificar rutas, nombres de filtros o estructura HTML.
- Cloudflare puede cambiar sus reglas y exigir una intervención no automática.
- La extracción de PDF depende de que el documento tenga texto seleccionable.
- `get_jurisprudencia_pdf_text` devuelve una ventana limitada por `maxChars`; si `truncated` es verdadero, no debe tratarse como documento completo.
- El MCP no interpreta, resume ni evalúa jurídicamente los fallos.
- La información recuperada debe validarse contra la fuente oficial y la versión vigente del expediente.